

Sistemas Operacionais

Lista de Exercícios 2

Jeann Victor
3 de novembro de 2025

Disciplina: Sistemas Operacionais

Assuntos: Gerenciamento de processos e Impasses.

Fonte: Sistemas Operacionais: Conceitos e Mecanismos

Autor do Material: Prof. Carlos A. Maziero

1 Questão 1

Explique, com suas palavras, a diferença entre programa, processo e thread. Em seguida, descreva uma situação do cotidiano que poderia ser usada como analogia para explicar esses três conceitos a uma pessoa leiga.

Programa: Seria o conjunto de sequências de uma ou mais instruções que ficam armazenados no disco rígido, uma aplicação parada. É o arquivo em si, o software instalado.

Processo: É a aplicação em atividade. É basicamente quando você pega aquele programa que estava só no disco e coloca para rodar. O processo é a "instância viva" do programa, é ele funcionando de verdade, usando a memória RAM e o processador.

Thread: Dentro de um processo que está rodando, existem várias linhas de execução, que são as threads. Cada thread é como um braço do processo, executando uma tarefa específica para ele. Várias threads podem estar fazendo coisas diferentes ao mesmo tempo, mas tudo dentro do mesmo processo.

O programa é a receita no livro. Ela está guardada, parada, só com as instruções.

- O processo é o ato de cozinhar, de seguir a receita. É quando você pega a receita, separa uma panela, os ingredientes e bota a mão na massa. É a execução completa daquela receita.
- A thread são as tarefas específicas dentro do ato de cozinhar. Por exemplo: uma thread fica responsável por picar os legumes, enquanto

outra thread fica cuidando do cozimento do arroz ao mesmo tempo. Ambas estão acontecendo dentro do mesmo "processo de cozinhar o jantar".

2 Questão 2

Um processo passa pelos estados "Novo → Pronto → Execução → Bloqueado → Terminado". Desenhe um diagrama de transições de estado e descreva em que situações reais (exemplo: leitura de arquivo, espera de entrada do usuário, preempção) cada transição ocorreria.

O estado Novo é quando um processo está começando a carregar seus recursos, e afins.

O estado Pronto é quando o processo já carregou tudo que precisava, ou seja, só precisa do acesso ao processador para começar a ser executado.

O estado Execução é quando o processo já está em execução dentro do processador.

O estado Bloqueado é quando o processo estava em execução, porém precisou do acesso a algo que não estava disponível ainda, ou quando seu tempo de preempção se esgotou; basicamente, o programa perde o seu acesso ao processador e retorna à fila.

O estado Terminado é quando o processo realizou tudo que queria, e quando terminado, todos os recursos utilizados são devolvidos.

3 Questão 3

Explique o que é uma troca de contexto e por que ela é necessária. Depois, analise: por que trocas de contexto muito frequentes podem degradar o desempenho do sistema?

Uma troca de contexto é o ato de suspender uma tarefa e reativar outra. Ela é de extrema importância pois existem momentos em que um processo requisita recursos que não estão disponíveis e que demorariam para conseguir. Para que o processador não fique ocioso, entra a troca de contexto.

O desempenho do sistema ser degradado tem uma explicação: basicamente, o sistema, ao realizar uma troca de contexto, precisa salvar o estado em que o processo anterior estava e carregar o estado do novo processo; portanto, essas operações de salvar e carregar são "perda de tempo", pois o processador não está realizando nada útil. Portanto, se ocorrem muitas

trocas de contexto, o processador gasta mais tempo nessas operações do que nas operações realmente importantes.

4 Questão 4

Em sistemas UNIX, cada processo pode gerar processos filhos com `fork()` e substituí-los com `exec()`. Explique o funcionamento dessas chamadas e discuta as vantagens e desvantagens do modelo hierárquico de processos em relação a um modelo totalmente plano (sem hierarquia).

O `fork()` cria uma cópia exata do processo atual (o pai), gerando um processo filho. Já o `exec()` substitui o programa em execução no processo atual por um novo programa completamente diferente. A combinação clássica é usar `fork()` para criar um filho e então `exec()` dentro dele para rodar um novo programa.

A vantagem do modelo hierárquico sobre um modelo plano é o controle. A hierarquia permite que um pai gerencie seus filhos, espere por seu término e recolha seus recursos, evitando processos "órfãos" e mantendo o sistema organizado. Sem a hierarquia, o sistema operacional teria que gerenciar todos os processos diretamente, o que seria menos eficiente e mais complexo para controlar grupos de processos relacionados.

5 Questão 5

Compare os principais mecanismos de comunicação entre processos (pipes, filas de mensagens, memória compartilhada e sockets). Para cada um, cite um exemplo de aplicação real em que ele seria a melhor escolha.

Pipes: Canal de comunicação unidirecional entre dois processos. Ex: filtros sequenciais simples.

Fila de Mensagens: Mensagens estruturadas trocadas entre processos, mantidas no kernel. Ex: mensagens assíncronas estruturadas.

Memória Compartilhada: Área de memória acessível por múltiplos processos simultaneamente. Ex: velocidade máxima em dados voláteis.

Sockets: Comunicação bidirecional entre processos locais ou em redes. Ex: para comunicação flexível local ou em rede.

6 Questão 6

Um desenvolvedor afirma que "usar threads é sempre melhor do que usar processos, pois threads são mais leves". Discuta criticamente essa afirmação, apresentando situações em que threads podem ser problemáticas e situações em que são ideais.

Essa afirmação está errada ao dizer que sempre é melhor. A escolha depende do sistema que se quer implementar. Se o sistema precisa de maior robustez e segurança, o uso de processos é mais viável. Isso porque, em uma aplicação baseada em processos, um erro fica restrito à sua própria área de memória, sem afetar os outros. Já em um sistema que usa apenas threads – mesmo sendo mais rápido – se uma thread falhar, o problema pode se alastrar e derrubar toda a aplicação, já que todas compartilham o mesmo espaço de memória.

7 Questão 7

Explique o que é uma condição de corrida. Descreva um exemplo simples (pode ser em pseudocódigo ou em uma analogia) e explique como o uso de semáforos evita esse problema.

Uma condição de corrida acontece quando dois ou mais processos ou threads acessam e modificam um mesmo recurso ao mesmo tempo, sem coordenação adequada. Isso pode fazer com que o resultado final dependa da ordem em que as operações ocorreram – o que é indesejado, pois o resultado pode se tornar incorreto.

Um exemplo simples é o de uma conta bancária: imagine que a mesma conta receba dois depósitos simultâneos. Se os dois processos lerem o saldo antigo ao mesmo tempo e depois gravarem o novo valor, um dos depósitos pode "sumir" no resultado final.

Para evitar esse tipo de problema, usamos semáforos. O semáforo funciona como um "sinal de trânsito" ou uma "chave" que controla o acesso a uma região crítica do código – isto é, um trecho onde o recurso compartilhado é manipulado. Antes de um processo acessar o recurso, ele precisa "abaixar" o semáforo (ou seja, pedir permissão). Enquanto um processo estiver dentro da região crítica, o semáforo impede que outros entrem, garantindo exclusão mútua. Quando o processo termina, ele "libera" o semáforo, permitindo que outro prossiga.

8 Questão 8

Escolha um dos problemas clássicos da IPC (Jantar dos Filósofos, Leitores e Escritores, Produtor-Consumidor).

a) Explique a essência do problema.

O problema do Produtor-Consumidor está relacionado ao controle de acesso a um buffer compartilhado entre dois tipos de processos: os produtores, que inserem itens, e os consumidores, que retiram. A essência do problema é evitar que o produtor tente colocar algo quando o buffer está cheio, e que o consumidor tente retirar quando o buffer está vazio, mantendo o sistema em equilíbrio e evitando erros de acesso.

b) Proponha uma solução teórica, mencionando o mecanismo de sincronização que usaria.

Minha ideia de solução é usar semáforos para controlar o acesso e evitar conflitos. Teria três semáforos principais:

- Um chamado `mutex`, que garante que apenas um processo (produtor ou consumidor) acesse o buffer por vez;
- Um chamado `empty`, que indica quantos espaços vazios há no buffer;
- E um chamado `full`, que indica quantos itens estão disponíveis.

O produtor só consegue inserir se `empty > 0`, e o consumidor só retira se `full > 0`. Quando um processo não pode agir, ele fica bloqueado esperando o outro liberar espaço ou adicionar itens. Isso evita acessos simultâneos e mantém a ordem correta das operações.

c) Discuta possíveis limitações dessa solução em sistemas reais.

Em sistemas com muitos produtores e consumidores, pode ocorrer espera excessiva ou até inanição (quando um processo espera muito tempo sem conseguir acesso). Também há um custo de desempenho, já que cada operação de sincronização adiciona uma pequena demora na execução.

9 Questão 9

Dado o seguinte conjunto de processos com tempos de chegada e duração:

Processo	Chegada	Duração	Prioridade
P1	0	8	2
P2	1	4	1
P3	2	9	3
P4	3	5	2

a) Monte o gráfico de execução para os algoritmos FCFS e Round Robin (q=3).

FCFS

P1 0–8
P2 8–12
P3 12–21
P4 21–26

$$26/4 = 6,5$$

Round Robin

P1 0–3
P2 3–6
P3 6–9
P4 9–12
P1 12–15
P2 15–16
P3 16–19
P4 19–21
P1 21–23
P2 23–26

$$26/4 = 6,5$$

b) Calcule o tempo médio de espera para cada um.

Não há como calcular de maneira correta o tempo do Round-Robin, devido a não saber o tempo que cada troca de contexto ocorre...

c) Analise qual algoritmo seria mais justo nesse caso e por quê.

Para este caso, o algoritmo mais justo seria o FCFS, pois o RR apresenta várias trocas de contexto, fazendo com que o processador gaste muito tempo "inútil".

10 Questão 10

Defina o que é um impasse (deadlock) e explique as quatro condições necessárias para que ele ocorra. Depois, cite um exemplo real

(dentro ou fora da computação) que representa cada condição.

Um deadlock ocorre quando processos esperam indefinidamente por recursos que nunca serão liberados. As 4 condições necessárias são as seguintes: exclusão mútua, posse e espera, não preempção e espera circular.

Exclusão Mútua: Uma cadeira só pode ser utilizada por uma pessoa; enquanto Pessoa1 senta, a Pessoa2 tem que esperar, e vice-versa.

Posse e Espera: É quando um programa A precisa de um recurso B, mas já está em posse do A. E ao mesmo tempo um programa B precisa de um recurso A e está em posse do B.

Não preempção: É basicamente quando um programa já está em uso de um recurso e não pode abrir mão dele, pois, caso abrir, o programa vai "morrer".

Espera Circular: o processo A segura o recurso 1 e espera pelo recurso 2, enquanto o processo B segura o recurso 2 e espera pelo recurso 1.

11 Questão 11

Considere os seguintes processos e recursos:

- P1 usa R1 e precisa de R2
- P2 usa R2 e precisa de R3
- P3 usa R3 e precisa de R1

- a) Modele o problema em um grafo de alocação de recursos.
- b) Verifique se há um impasse.
- c) Explique o raciocínio e proponha uma forma de resolver o problema.

Não há resolução registrada para esta questão no material original.

12 Questão 12

Explique, em termos simples, a ideia por trás do algoritmo do banqueiro de Dijkstra. Depois, avalie: por que esse algoritmo é considerado impraticável em muitos sistemas modernos? Sugira uma situação em que ele seria útil.

Esse algoritmo faz uma analogia entre as tarefas de um sistema e os clientes de um banco, tratando os recursos como créditos emprestados às

tarefas para a realização de suas atividades. O banqueiro decide quais solicitações de empréstimo deve atender para conservar suas finanças em um estado seguro.

Alguns problemas decorrentes dessa estratégia são o custo de manutenção contínua do grafo de alocação e, sobretudo, o custo de sua análise: algoritmos de busca de ciclos em grafos têm custo computacional elevado, portanto sua ativação com muita frequência poderá prejudicar o desempenho do sistema. Por outro lado, se a detecção for ativada apenas esporadicamente, impasses podem demorar muito para ser detectados, o que também é ruim para o desempenho do sistema.

O algoritmo do banqueiro pode ser útil em sistemas embarcados críticos ou sistemas bancários com número limitado e previsível de tarefas, onde o controle de segurança e estabilidade é mais importante do que o desempenho.

13 Questão 13

Compare as estratégias de prevenção, evitação e detecção de impasses. Para cada uma, cite: um ponto forte, uma limitação prática, e um exemplo de uso em sistemas reais.

Não há resolução registrada para esta questão no material original.

14 Questão 14

Em um sistema Linux moderno, múltiplos processos compartilham CPU, memória e dispositivos de E/S. Discuta como os mecanismos de gerenciamento de processos, escalonamento e sincronização atuam juntos para evitar impasses e garantir desempenho. Conclua avaliando como as decisões de projeto do kernel influenciam diretamente a experiência do usuário.

Não há resolução registrada para esta questão no material original.